

01 · Tokenización

Primer paso del pipeline de minería de texto: dividir un texto en unidades más pequeñas llamadas *tokens* (frases, palabras o subpalabras). Todo lo que viene después —normalización, Bag of Words, TF-IDF— opera sobre estos tokens.

Corpus: las **reseñas de entrega** (`post_compra`) de la tienda-virtual, en `datos/resenas_entrega.csv` .

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")
```

```
def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)
```

```
resenas = cargar("resenas_entrega.csv")
print(f"{len(resenas)} reseñas")
resenas[["texto"]].head(5)
```

Leyendo ../datos/resenas_entrega.csv (15.0 KB)
110 reseñas

texto

- | | texto |
|---|---|
| 0 | Recibi el pedido incompleto, faltaba el cable ... |
| 1 | El segundo cargador Belkin llevo sin contratie... |
| 2 | La segunda Switch Lite que regale llevo perfec... |
| 3 | El segundo television TCL llevo perfecto, bien... |
| 4 | Me confirmaron por correo la garantia de un an... |

```
# El corpus con el que trabajaremos: la columna de texto libre
corpus = resenas[["texto"]].dropna().astype(str).tolist()
print(f"{len(corpus)} documentos")
print("Ejemplo:", corpus[0])
```

110 documentos

Ejemplo: Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja. Tuve que esperar un envio adicional solo para recibir ese accesorio.

Tokenización por frases (NLTK)

`nltk.sent_tokenize` parte un texto en oraciones usando un modelo entrenado para el idioma (puntos, signos, abreviaturas). Tomamos la reseña más larga del corpus para ver varias frases.

```
import nltk

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

texto_largo = max(corpus, key=len)
frases = nltk.sent_tokenize(texto_largo, language="spanish")

print(texto_largo, "\n")
for i, f in enumerate(frases, 1):
    print(f"Frase {i}: {f}")
```

El SSD Samsung T7 llego con el cable USB-C incorrecto dentro de la caja, uno que no era compatible con el puerto del disco. Tuve que comprar un cable aparte mientras resolvian el reclamo.

Frase 1: El SSD Samsung T7 llego con el cable USB-C incorrecto dentro de la caja, uno que no era compatible con el puerto del disco.

Frase 2: Tuve que comprar un cable aparte mientras resolvian el reclamo.

Tokenización por palabras (NLTK)

`nltk.word_tokenize` separa cada frase en palabras y signos de puntuación como tokens independientes.

```
tokens_por_frase = [nltk.word_tokenize(f, language="spanish") for f in frases]
for i, toks in enumerate(tokens_por_frase, 1):
    print(f"Frase {i}: {toks}")
```

Frase 1: ['El', 'SSD', 'Samsung', 'T7', 'llego', 'con', 'el', 'cable', 'USB-C', 'incorrecto', 'dentro', 'de', 'la', 'caja', ',', 'uno', 'que', 'no', 'era', 'compatible', 'con', 'el', 'puerto', 'del', 'disco', '.']

Frase 2: ['Tuve', 'que', 'comprar', 'un', 'cable', 'aparte', 'mientras', 'resolvian', 'el', 'reclamo', '.']

```
from collections import Counter

# Tokenizamos TODO el corpus y miramos el vocabulario
todos = [t.lower() for texto in corpus for t in nltk.word_tokenize(texto,
language="spanish")]
vocabulario = sorted(set(todos))

print(f"Tokens totales : {len(todos)}")
print(f"Vocabulario : {len(vocabulario)} tokens únicos")
print(f"\n20 tokens más frecuentes (todavía sin normalizar):")
for palabra, n in Counter(todos).most_common(20):
    print(f" {palabra!r:15} {n}")
```

Tokens totales : 2322
Vocabulario : 609 tokens únicos

20 tokens más frecuentes (todavía sin normalizar):

'.'	146
'el'	136
'la'	108
'de'	105
'llego'	64
'con'	59
'en'	53
','	48
'que'	47
'un'	34
'a'	27
'sin'	25
'y'	25
'del'	22
'se'	22
'caja'	18
'por'	18
'bien'	17
'fue'	16
'no'	16

Casi todos los tokens más frecuentes son signos de puntuación y *stopwords* (el , de , la , y ...).
Ese ruido es exactamente lo que elimina el paso de **normalización** (02_normalizacion.ipynb).

spaCy: tokenización con información lingüística

spaCy tokeniza y, en la misma pasada, anota cada token: categoría gramatical (pos_), lema, si es puntuación, si es *stopword*, etc.

```
import spacy

nlp = spacy.load("es_core_news_sm")

doc = nlp(corpus[0])
tabla = pd.DataFrame(
    {
        "token": [t.text for t in doc],
        "lema": [t.lemma_ for t in doc],
        "pos": [t.pos_ for t in doc],
        "es_stopword": [t.is_stop for t in doc],
        "es_puntuacion": [t.is_punct for t in doc],
    }
)
tabla.head(20)
```

	token	lema	pos	es_stopword	es_puntuacion
0	Recibi	Recibi	PROPN	False	False
1	el	el	DET	True	False
2	pedido	pedido	NOUN	False	False
3	incompleto	incompleto	NOUN	False	False
4	,	,	PUNCT	False	True
5	faltaba	faltar	VERB	False	False
6	el	el	DET	True	False
7	cable	cable	NOUN	False	False
8	de	de	ADP	True	False
9	carga	carga	NOUN	False	False
10	original	original	ADJ	False	False
11	del	del	ADP	True	False
12	OnePlus	OnePlus	PROPN	False	False
13	12	12	NUM	False	False
14	dentro	dentro	ADV	True	False
15	de	de	ADP	True	False
16	la	el	DET	True	False
17	caja	caja	NOUN	False	False
18	.	.	PUNCT	False	True
19	Tuve	tener	VERB	False	False

Subpalabras (BPE / WordPiece / SentencePiece)

Los modelos basados en Transformers (GPT, BERT, T5) no tokenizan por palabra sino por **subpalabra**: fragmentos frecuentes que permiten representar términos fuera de vocabulario combinando piezas conocidas. Un tokenizador de palabras trata "OnePlus", "iPhone" o "repuesto" como unidades atómicas; uno de subpalabras las parte en One, Plus, ... y nunca se queda sin representación.

En este curso trabajamos a nivel de palabra (NLTK / spaCy), suficiente para Bag of Words y TF-IDF.

Siguiente: 02_normalizacion.ipynb — reducir estos tokens a una forma canónica (minúsculas, sin signos, sin stopwords, lematizados).